Data Sciencetech Institute (DSTI)

Cybersecurity Attack Type Detection

End-to-End Machine Learning Pipeline and Deployment Report

Authors:
Nassim Chakroun
Moncef Halleb
Firas Abidhiaf
Abderrahman Ammar
Mongi Selim Sahli

Instructor:
Hanna Abi Akl

February 28, 2026

GitHub: https://github.com/nassimch112/CybersecML

Contents

1 Introduction **2**

2 Methodology **2**

2.1 Pipeline Overview 2

2.2 Exploratory Data Analysis 2

2.3 Feature Engineering and Selection 4

2.4 Target Engineering and Label Construction 5

2.5 Model Selection and Evaluation Strategy 6

3 Results **6**

3.1 Comparative Performance Analysis 6

3.2 Interpretation of Findings 7

4 Deployment **7**

5 Conclusion **10**

1 Introduction

This project addresses the challenge of multiclass cyber-attack detection—specifically DDoS, Intrusion, and Malware—within synthetic network telemetry. The dataset comprises 40,000 records featuring mixed categorical and numerical attributes like protocols, packet lengths, and device metadata. The objective is to build a robust, reproducible pipeline that bridges the gap between raw telemetry and real-time deployment via a web-based inference tool.

2 Methodology

2.1 Pipeline Overview

The implemented pipeline follows these phases:

1. Data loading and baseline cleaning
2. Exploratory Data Analysis (EDA)
3. Feature engineering and feature matrix construction
4. Target strategy (original target and engineered target perspectives)
5. Model training, tuning, and comparative evaluation
6. Deployment as a web application for single and batch inference

2.2 Exploratory Data Analysis

EDA was performed to profile class distributions, categorical composition, numeric behavior, temporal patterns, and protocol-target relationships. This step informed later feature design and interpretation.

EDA Artifacts

Figure 1: Target distribution (count and share) from EDA section.

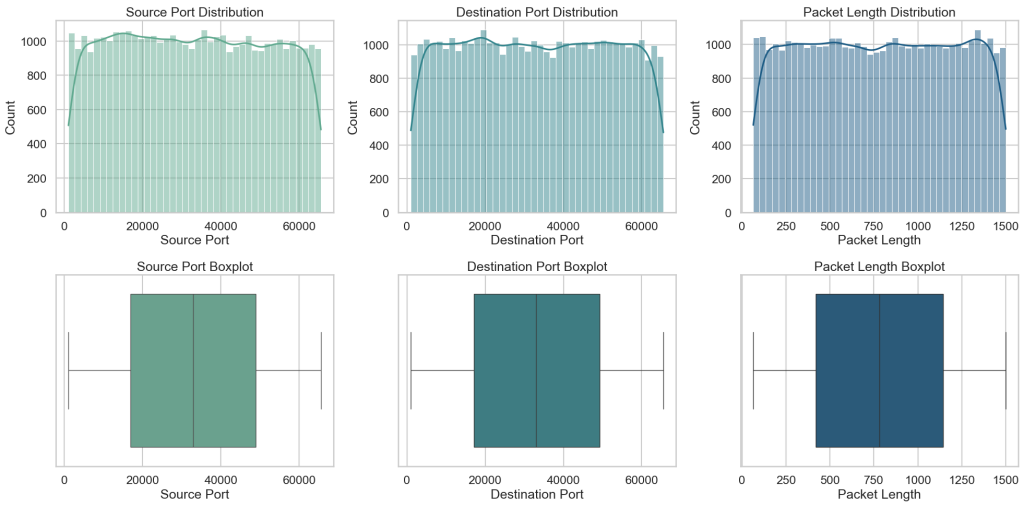


Figure 2: Numeric distributions: Source Port, Destination Port, and Packet Length.

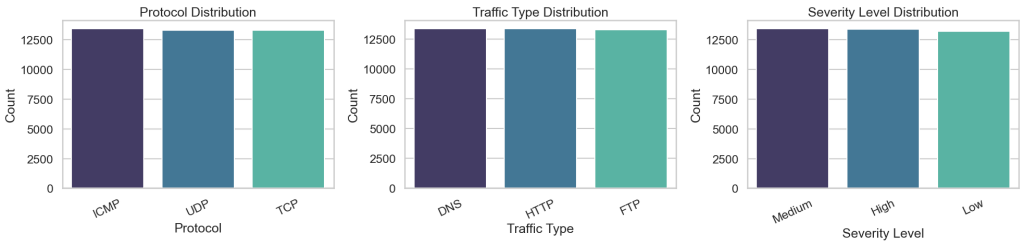


Figure 3: Categorical landscape of Protocol, Traffic Type, and Severity levels.

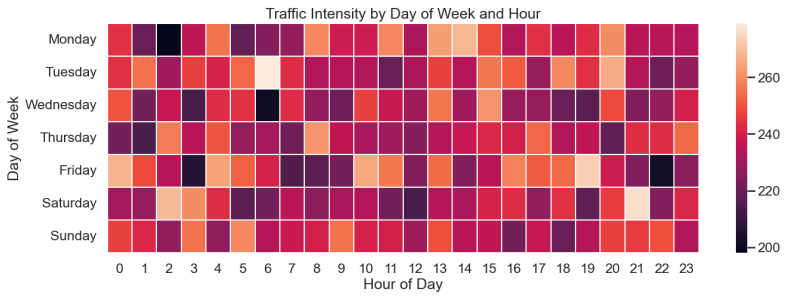


Figure 4: Temporal activity heatmap showing traffic intensity by day and hour.

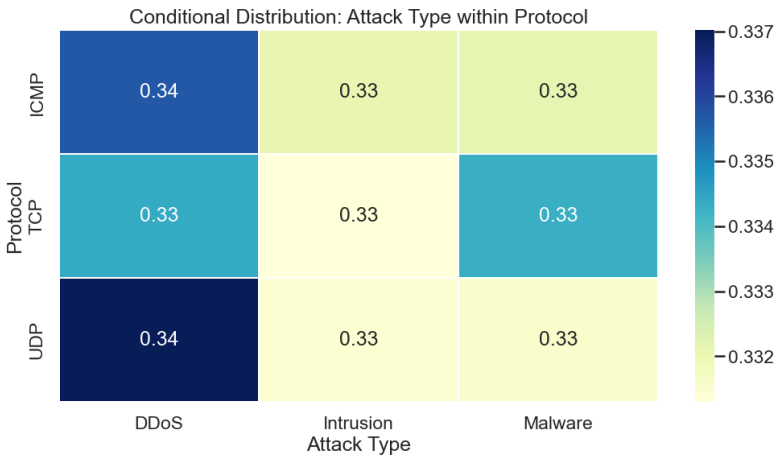


Figure 5: Conditional attack distribution by protocol heatmap.

EDA Summary

EDA indicated broadly balanced synthetic distributions across major categories and weak direct separation in isolated raw variables. This motivated a stronger feature engineering strategy focused on interactions and transformations instead of relying on single-column discrimination.

2.3 Feature Engineering and Selection

Feature engineering was the core technical contribution, expanding the raw telemetry into a high-dimensional space of 42+ features. These were organized into five primary signal families:

- **Temporal Dynamics:** Derived from timestamps to capture cyclical attack patterns, including *hour_bin*, *day_of_week*, *is_weekend*, and *is_off_hours*.
- **Network Topology:** IP-based features including the extraction of the first three octets (*src_ip_octet1-3*) to identify subnet-level commonalities and port-relationship

flags such as *is_web_port*.

- **Payload & Packet Metadata:** Advanced ratios and logs, specifically *payload_to_packet_ratio*, *packet_len_log*, and binary flags for Indicators of Compromise (IoC).
- **Device & User-Agent Intelligence:** Detailed parsing of User-Agent strings to extract *os_family*, *browser_family*, *ua_token_count*, and *is_mobile* indicators.
- **Categorical Encoding:** Transformation of high-cardinality strings (Protocol, Traffic Type) into a machine-readable format via one-hot encoding, ensuring the model captures non-linear relationships.

This multifaceted approach allowed the LightGBM model to identify complex "fingerprints" of DDoS, Intrusion, and Malware activity that were not visible in the raw, isolated variables.

2.4 Target Engineering and Label Construction

A significant challenge identified during initial analysis was the lack of logical consistency in the original synthetic labels. To create an operationally relevant model, we implemented a rule-based target engineering strategy.

- **Rationale:** The original labels often contradicted standard network behavior. By engineering labels, we transformed the task into a "logic-engine" validation, ensuring the model learns actual cybersecurity heuristics rather than noise.
- **Implementation:** We developed a function, `generate_target_from_raw`, to assign labels based on high-confidence domain indicators:
  - **DDoS:** Flagged via volumetric patterns, port flooding, and specific TCP/UDP protocol signatures.
  - **Intrusion:** Triggered by "Exploit" or "Privilege" traffic types and anomalous port-to-payload ratios.
  - **Malware:** Identified by known Indicators of Compromise (IoC) and User-Agent anomalies found during EDA.
- **Output:** This yielded a consistent 3-class target (*DDoS, Intrusion, Malware*), providing a reliable ground truth for the LightGBM pipeline.

2.5 Model Selection and Evaluation Strategy

With the engineered labels established, we utilized a LightGBM classifier. This choice was driven by its ability to handle the 42+ engineered features efficiently. To address the discrepancy between label sets, we performed cross-target benchmarking, which revealed a significant performance gap (96% vs 33%)—further justifying the need for the engineered approach.

3 Results

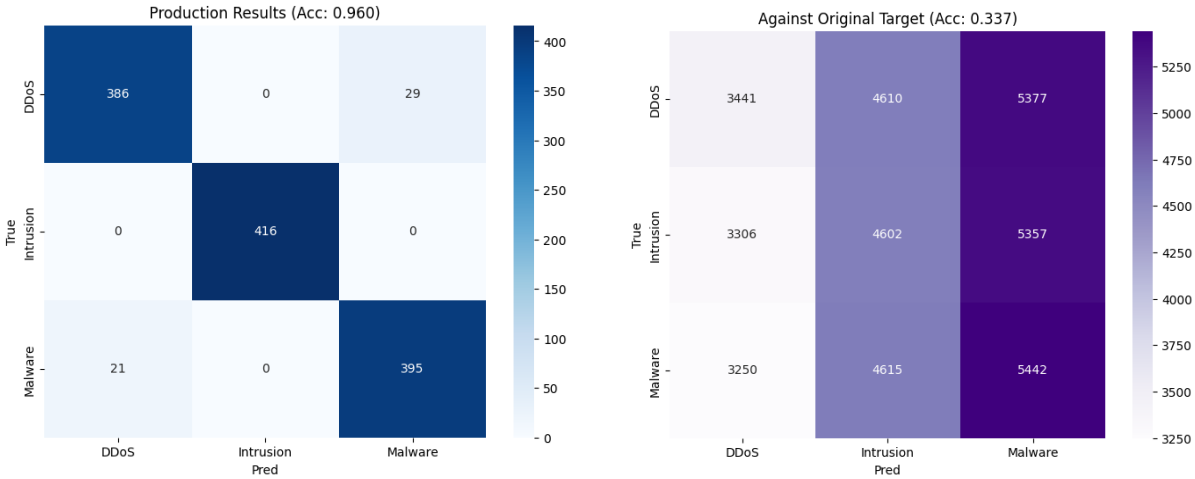
3.1 Comparative Performance Analysis

The model was evaluated using two distinct perspectives to measure its robustness. The divergence in scores (as shown in Table 1) highlights that while the model is an excellent learner of network logic, the original synthetic labels do not follow that same logic.

Table 1: Comparison of Model Performance across Target Perspectives

Target Set	**Accuracy**	**Macro F1**	**Inference**
Engineered Labels	**0.9599**	**0.96**	High Logic Alignment
Original Target Labels	0.3372	0.33	Low Label Consistency

The confusion matrices in Figure 6a and Figure 6b further illustrate this. While Figure 6a shows a nearly diagonal matrix (indicating high precision), Figure 6b shows significant overlap, suggesting the original labels do not follow the network telemetry features consistently.



(a) Confusion Matrix (Engineered)

(b) Confusion Matrix (Original)

Figure 6: Visualizing the classification discrepancy between target types.

3.2 Interpretation of Findings

The high accuracy on engineered labels confirms that the feature engineering successfully captured the "fingerprints" of the attack types. The low accuracy on the original target suggests that the synthetic dataset's initial labels were likely generated using a different (or random) logic, which our model correctly identified as inconsistent with the network telemetry features.

4 Deployment

The model was operationalized using a Streamlit web application. The UI supports:

- **Manual Inference:** Real-time prediction based on user-editable form inputs.
- **Batch Processing:** CSV upload capability with downloadable prediction results.
- **Diagnostics:** Visual probability breakdowns and confidence metrics for each prediction.

Application capabilities

- **Manual mode:** single-event prediction with editable network/payload/device inputs.
- **Preset profiles:** one-click form presets for DDoS/Intrusion/Malware-style scenarios.

- **CSV mode:** batch inference on uploaded files with downloadable prediction output.
- **Dual comparison panel:** side-by-side *Predicted* vs *Generated* outcomes.
- **Diagnostics:** distribution and confidence summaries for uploaded workloads.

Deployment artifacts

- Serialized model bundle (PKL) with feature schema and class metadata.
- Streamlit interface script for inference and analytics.
- Dependency manifest for reproducible environment setup.

Web UI

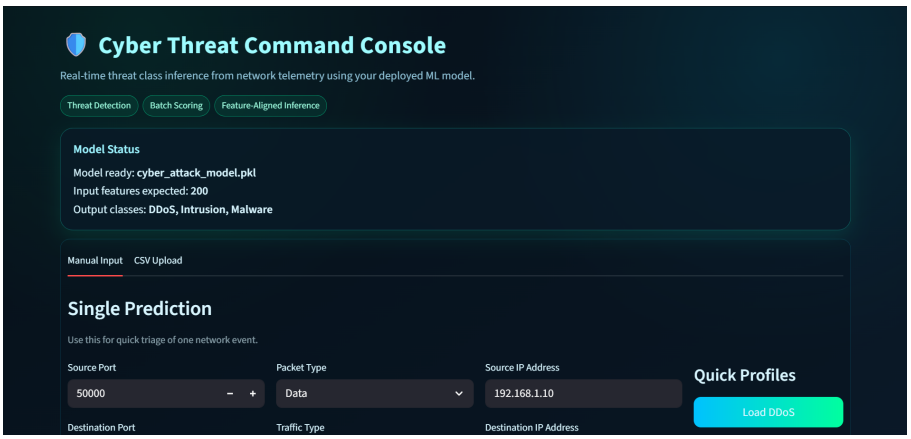


Figure 7: Web application dashboard and model operational status.

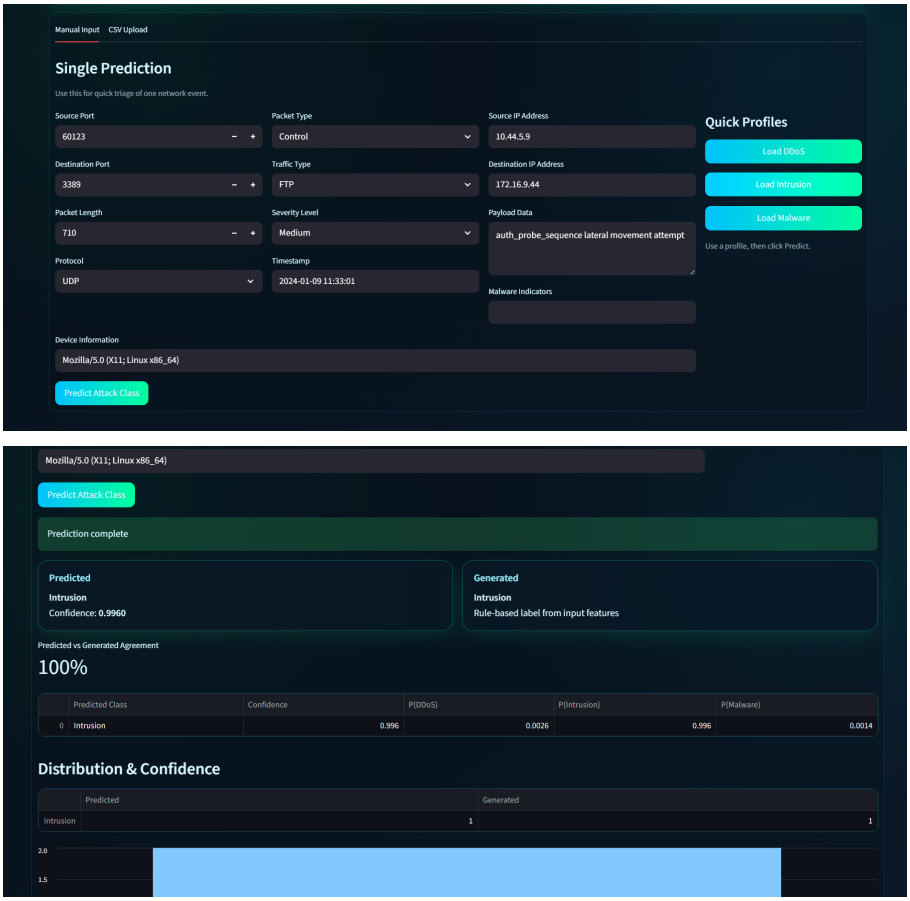


Figure 8: Manual prediction interface with preset profile auto-fill and result card.

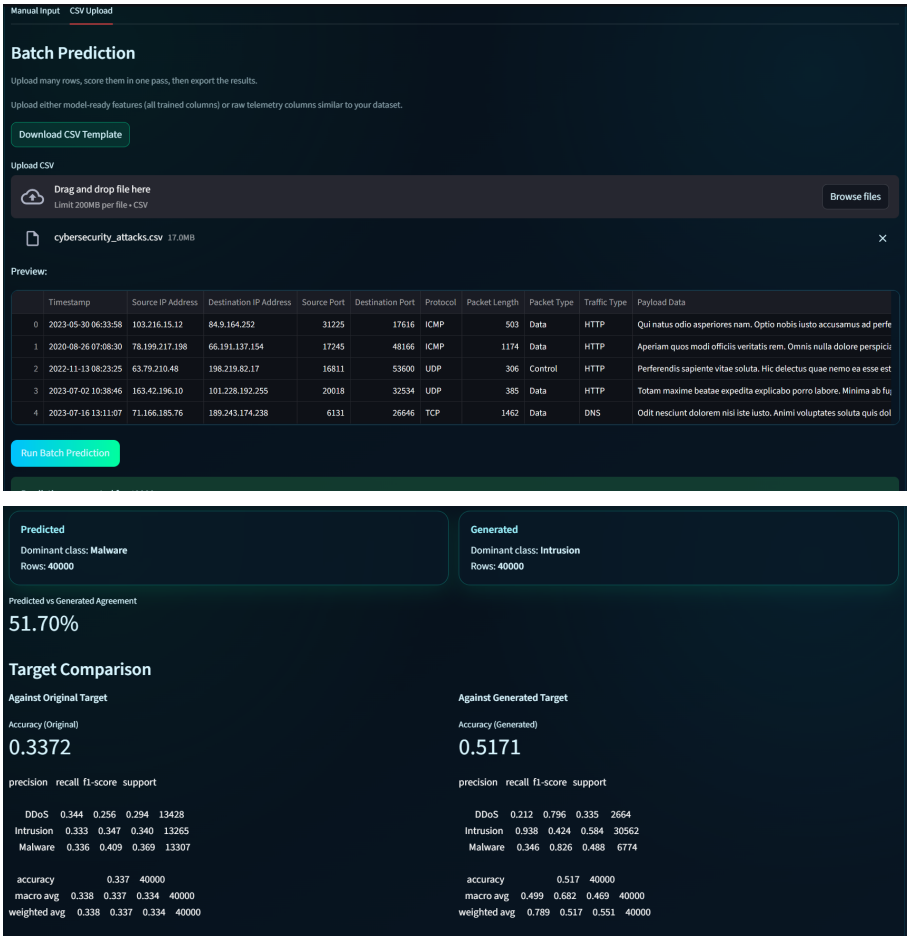


Figure 9: CSV batch prediction workflow and side-by-side target comparison panel.

Figure 10: Distribution analysis and model confidence diagnostics for uploaded workloads.

5 Conclusion

This project demonstrates a successful end-to-end ML workflow. Key outcomes include a high-performance feature space and an interactive deployment tool. Future work will

focus on:

- Formalizing the feature engineering as a standalone API.
- Implementing uncertainty analysis for better confidence calibration.
- Implementing a formal Feature Store or automated pipeline trigger (e.g., via GitHub Actions) to ensure the `generate_target_from_raw` logic stays synced between the training notebook and the Streamlit app.

End of Report